Практическая работа №3

Индексы

Для того, чтобы оптимизировать поиск в базе данных зачастую используют индексы. Индексы создаются с помощью конструкции *CREATE index name_index on name_table (column_name_table)*, где

name_index – имя индекса;

name_table – имя таблицы, в которой создается индекс;

column_name_table – имя столбца, в котором создается индекс.

Индекс – это отсортированный набор значений, который формирует карту поиска (B-дерево). B-дерево – это сбалансированное дерево поиска, в котором каждый узел содержит множество ключей и имеет более двух потомков. Поиск работает следующим образом:

Шаг 1: Считать элемент для поиска.

Шаг 2: Сравнить искомый элемент с первым значением ключа в корневом узле дерева.

Шаг 3: Если они совпадают, вывести: «Искомый узел найден!» и завершить поиск.

Шаг 4: Если они не совпадают, проверить больше или меньше значение элемента, чем текущее значение ключа.

Шаг 5: Если искомый элемент меньше, продолжить поиск по левому поддереву.

Шаг 6: Если искомый элемент больше, сравнить элемент со следующим значением ключа в узле и повторять Шаги 3, 4, 5 и 6 пока не будет найдено совпадение или пока искомый элемент не будет сравнен с последним значением ключа в узле-листе.

Шаг 7: Если последнее значение ключа в узле-листе не совпало с искомым, вывести «Элемент не найден!» и завершить поиск.

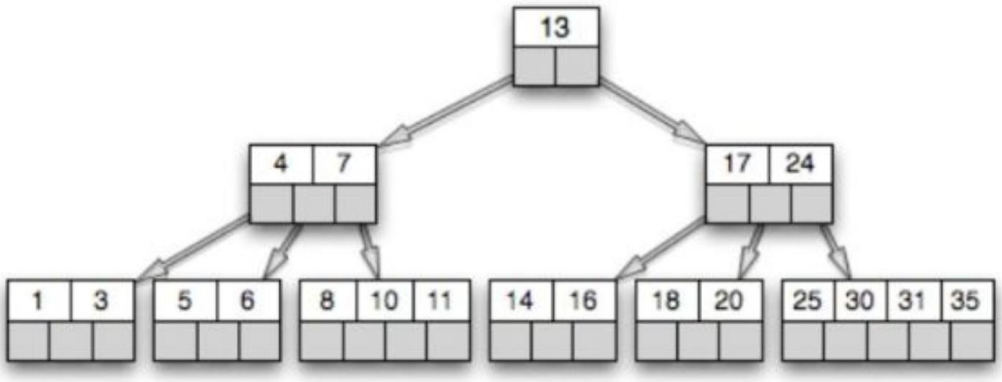


Рисунок 1 – Пример B-дерева

Пример.

Создать индекс для столбца даты вылета, чтобы оптимизировать поиск по данному полю.

```
CREATE INDEX date_avia on avia (date_avia)
```

Далее следует запрос на выборку по индексированному полю:

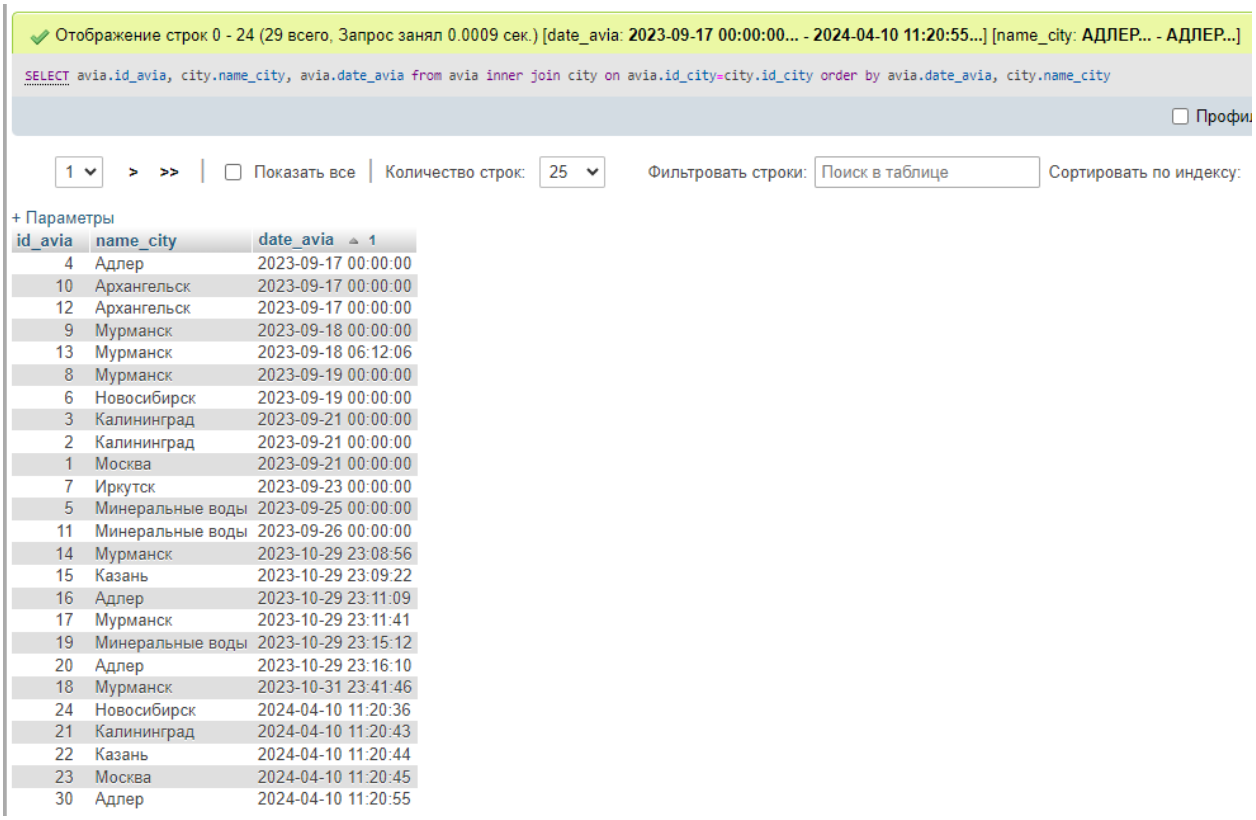
```
SELECT * FROM `avia` WHERE date_avia like '2023%'
```

Показать все | Количество строк: 25 Фильтров

+ Параметры

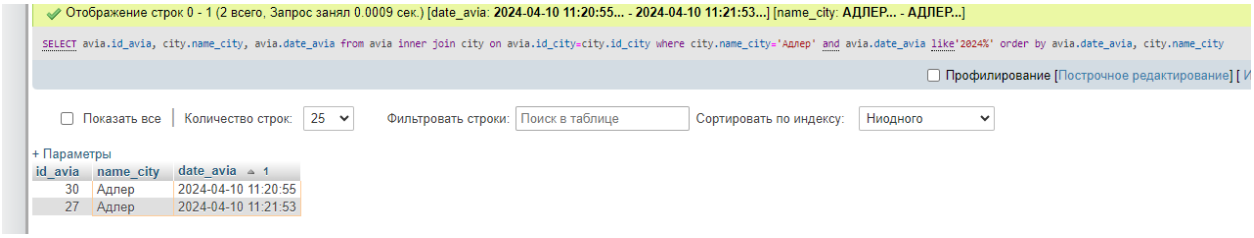
id_avia	id_city	id_air	date_avia
1	1	2	2023-09-21 00:00:00
2	2	3	2023-09-21 00:00:00
3	2	4	2023-09-21 00:00:00
4	4	2	2023-09-17 00:00:00
5	5	2	2023-09-25 00:00:00
6	6	2	2023-09-19 00:00:00
7	7	3	2023-09-23 00:00:00
8	8	1	2023-09-19 00:00:00
9	8	3	2023-09-18 00:00:00
10	9	5	2023-09-17 00:00:00
11	5	4	2023-09-26 00:00:00
12	9	3	2023-09-17 00:00:00
13	8	1	2023-09-18 06:12:06
14	8	1	2023-10-29 23:08:56
15	3	2	2023-10-29 23:09:22
16	4	2	2023-10-29 23:11:09
17	8	3	2023-10-29 23:11:41
18	8	3	2023-10-31 23:41:46
19	5	2	2023-10-29 23:15:12
20	4	2	2023-10-29 23:16:10

Далее был создан индекс для столбца названия города для оптимального поиска по городу назначения. В текущем примере выборка без использования фильтров:



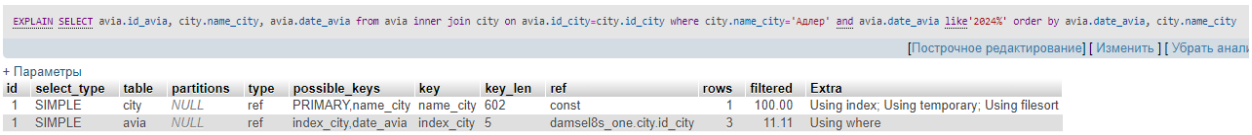
id_avia	name_city	date_avia 1
4	Адлер	2023-09-17 00:00:00
10	Архангельск	2023-09-17 00:00:00
12	Архангельск	2023-09-17 00:00:00
9	Мурманск	2023-09-18 00:00:00
13	Мурманск	2023-09-18 06:12:06
8	Мурманск	2023-09-19 00:00:00
6	Новосибирск	2023-09-19 00:00:00
3	Калининград	2023-09-21 00:00:00
2	Калининград	2023-09-21 00:00:00
1	Москва	2023-09-21 00:00:00
7	Иркутск	2023-09-23 00:00:00
5	Минеральные воды	2023-09-25 00:00:00
11	Минеральные воды	2023-09-26 00:00:00
14	Мурманск	2023-10-29 23:08:56
15	Казань	2023-10-29 23:09:22
16	Адлер	2023-10-29 23:11:09
17	Мурманск	2023-10-29 23:11:41
19	Минеральные воды	2023-10-29 23:15:12
20	Адлер	2023-10-29 23:16:10
18	Мурманск	2023-10-31 23:41:46
24	Новосибирск	2024-04-10 11:20:36
21	Калининград	2024-04-10 11:20:43
22	Казань	2024-04-10 11:20:44
23	Москва	2024-04-10 11:20:45
30	Адлер	2024-04-10 11:20:55

Видно, что и после применения фильтров время обработки запроса не изменилось.



id_avia	name_city	date_avia 1
30	Адлер	2024-04-10 11:20:55
27	Адлер	2024-04-10 11:21:53

Стоит отметить, что все атрибуты, которым был назначен **первичный ключ** (PRIMARY KEY) по умолчанию являются **индексами**. Для того, чтобы понять механику выполнения запроса можно воспользоваться оператором **EXPLAIN**.



id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	city	NULL	ref	PRIMARY,name_city	name_city	602	const	1	100.00	Using index; Using temporary; Using filesort
1	SIMPLE	avia	NULL	ref	index_city,date_avia	index_city	5	damsel8s_one.city.id_city	3	11.11	Using where

Где значения столбцов таблицы:

Id – это порядковый номер каждого запроса на выборку;

Select_type – указывает на тип подзапроса, в текущем случае это SIMLE (простой запрос), но могут встречаться и другие выражения, например, SUBqueRY (для вложенных запросов), Derived (когда в блок FROm вложен подзапрос, используется при применении временных таблиц). Union (при использовании UNION первый считается частью внешнего запроса).

Table – имя таблицы.

Partitions – показывает какой партиции соотвествуют даты запроса.

Type – указывает каким образом данные выбираются из таблиц. Может принимать значения: ALL. Обычно речь идет о полном сканировании таблицы, то есть MySQL будет

просматривать строчку за строчкой, если только в запросе нет LIMIT или в колонке extra не указано Distinct/not exists.

Index. В этом случае MySQL тоже просматривает таблицу целиком, но в порядке, заданном индексом. В этом случае не требуется сортировка, но строки выбираются в хаотичном порядке. Лучше, если в колонке extra будет указано using index, что означает, что вместо полного сканирования таблицы, MySQL проходит по дереву индексов. Такое происходит, когда удалось использовать покрывающий индекс. Range. Индекс просматривается в заданном диапазоне. Поиск начинается в определенной точке индекса и возвращает значения, пока истинно условие поиска. range может быть использован, когда проиндексированный столбец сравнивается с константой с использованием операторов =, <>, >, >=, <, <=, IS_NULL, <=>, BETWEEN, LIKE или IN. Index_subquery. Вы увидите это значение, если в операторе IN есть подзапрос, для которого оптимизатор MySQL смог использовать поиск по индексу. Uindex_merge. Если оптимизатор использовал range-сканирование для нескольких таблиц, он может объединить их результаты. В зависимости от метода слияния, поле extra примет одно из следующих значений: Using intersect — пересечение, Using union — объединение, Using sort_union — объединение сортировки слияния. Unique_subquery. Похож на index_subquery, но, для подзапроса используется уникальный индекс, такой как Primary key или Unique index. Index_merge. Если оптимизатор использовал range-сканирование для нескольких таблиц, он может объединить их результаты. В зависимости от метода слияния, поле extra примет одно из следующих значений: Using intersect — пересечение, Using union — объединение, Using sort_union — объединение сортировки слияния. Ref_or_null. Этот случай похож на ref, за исключением того, что MySQL будет выполнять второй просмотр для поиска записей, содержащих NULL- значения. Fulltext. Использование FULLTEXT-индекса. Ref. Поиск по индексу, в результате которого возвращаются все строки, соответствующие единственному заданному значению. Применяется в случаях, если ключ не является уникальным, то есть не Primary key или Unique index , либо используется только крайний левый префикс ключа. Ref может быть использован только для операторов = или <=>. eq_ref. Считывается всего одна строка по первичному или уникальному ключу. Работает только с оператором =. Справа от знака “=” может быть константа или выражение. const. Таблица содержит не более одной совпадающей строки. Если при оптимизации MySQL удалось привести запрос к константе, то столбец type будет равен const. Например, если вы ищете что-то по первичному ключу, то оптимизатор может преобразовать значение в константу и исключить таблицу из соединения JOIN. system. В таблице только одна строка. Частный случай const. NULL. Означает, что оптимизатор смог сам найти нужные данные для запроса и обращаться к таблице не требуется. Например, для выборки минимального значения проиндексированного, в порядке возрастания столбца достаточно выбрать первое значение из индекса.

Possible_keys

Этот столбец не зависит от порядка таблиц, отображаемых EXPLAIN, поскольку список создается на ранних этапах оптимизации. Если в столбце значение NULL, то соответствующих индексов не нашлось.

Keys

Указывает на ключ (индекс), который оптимизатор MySQL решил фактически использовать. Может вообще отсутствовать в столбце possible_keys. Так бывает в случаях, когда ни один из ключей в possible_keys не подходит для поиска строк, но все столбцы, выбранные запросом, являются частью какого-то другого индекса. Тогда оптимизатор может задействовать эти столбцы для поиска по покрывающему индексу, так как сканирование индекса эффективнее, чем сканирование таблицы целиком.

Key_len

Показывает длину выбранного ключа (индекса) в байтах. Например, если у вас есть primary key id типа int, то, при его использовании, key_len будет равен 4, потому что длина int всегда равна 4 байта. В случае составных ключей key_len будет равен сумме байтов их типов. Если столбец key равен NULL, то значение key_len так же будет NULL.

Ref

Показывает, какие столбцы или константы сравниваются с указанным в key индексом. Принимает значения NULL, const или название столбца другой таблицы. Возможно значение func, когда сравнение идет с результатом функции. Чтобы узнать, что это за функция, можно после EXPLAIN выполнить команду SHOW WARNINGS.

Rows

Показывает количество строк, которое, по мнению MySQL, будет прочитано. Это число является приблизительным и может оказаться очень неточным. Оно вычисляется при каждой итерации плана выполнения с вложенными циклами. Часто это значение путают с количеством строк в результирующем наборе, что неверно, потому что столбец rows показывает количество строк, которые нужно будет просмотреть. При вычислении значения не учитываются буферы соединения и кеши (в том числе кеши ОС и оборудования), поэтому реальное число может быть намного меньше предполагаемого.

Filtered

Показывает, какую долю от общего количества числа просмотренных строк вернет движок MySQL. Максимальное значение 100, то есть будет возвращено все 100 % просмотренных строк. Если умножить эту долю на значение в столбце rows, то получится приблизительная оценка количества строк, которые MySQL будет соединять с последующими таблицами. Например, если в строке rows 100 записей, а значение filtered 50,00 (50 %), то это число будет вычислено как 100 x 50 % = 50.

extra

В этом столбце собрана различная дополнительная информация. Ниже я разберу все возможные значения. Наиболее важные из них будут помечены (!!!). Такие значения не всегда означают, что это плохо или хорошо, но точно стоит обратить на них внимание.

const row not found. Для запроса, вида SELECT … FROM table, таблица table оказалась пустая.

Deleting all rows. Некоторые движки MySQL, такие как MyISAM, поддерживают методы быстрого удаления всех строк из таблицы. Если механизм удаления поддерживает эту оптимизацию, то значение Deleting all rows будет значением в столбце Extra.

Distinct. Если в запросе присутствует DISTINCT, то MySQL прекращает поиск, после нахождения первой подходящей строки.

Составные индексы.

Логично использовать составные индексы для поиска сразу по двум связным полям. Например, если сделаны отдельно колонки «фамилия» и «имя», то можно сделать составной индекс, который сможет улучшить индексацию.

```
create index name_birth on passengers (name_pass, date_of_birth)
```

Недостатки использования индексов.

Каждый индекс представляет собой таблицу (пусть и особый тип таблицы, но все же это таблица). Следовательно, каждый раз, когда строка добавляется в таблицу или удаляется из неё, должны быть изменены все индексы в этой таблице. При обновлении строки любые индексы для столбца (или столбцов), которые были затронуты, также должны быть изменены. Следовательно, чем больше у вас индексов, тем больше должна работать СУБД, чтобы поддерживать все объекты схемы в актуальном состоянии — что приводит к замедлению работы.

Задание.

Создать индексированные поля согласно предметной области (составные и одинарные). Указать аналитику запроса с помощью EXPLAIN – использовать, как и вложенные запросы, так и запросы с UNION, чтобы отразить разные значения столбцов.